

Lab 13 – Code Refactoring Using AI Assistance

Task 1 – Removing Code Duplication

Prompt

1. Refactor the given legacy Python code that calculates the area and perimeter of rectangles many times.
2. The code currently repeats the same calculation logic again and again which makes the program longer and harder to maintain.
3. Create a reusable function that accepts length and width of a rectangle as parameters.
4. The function should calculate and print the area and perimeter for each rectangle.
5. Make sure the program becomes shorter, reusable, and easy to understand without repeating the same code.

Code:

```
1
2 def rectangle_details(length, width):
3     """
4     This function calculates and prints
5     the area and perimeter of a rectangle
6     """
7     area = length * width
8     perimeter = 2 * (length + width)
9
10    print("Area of Rectangle:", area)
11    print("Perimeter of Rectangle:", perimeter)
12
13
14    rectangle_details(5, 10)
15    rectangle_details(7, 12)
16    rectangle_details(10, 15)
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Python\Python311\python.exe C:\Users\rahul\Desktop\AI Assistanting coding\lab-13.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. A function called `rectangle_details()` is created to remove repeated code.
2. The function takes length and width as inputs.
3. It calculates the area and perimeter inside the function.
4. This makes the program shorter and easier to reuse.

Task 2 – Optimizing Loops and Conditionals

Prompt

1. The given Python program checks if some names exist in a list using nested loops.
2. Nested loops make the program slower when the list becomes very large.
3. Refactor the code using a Python set for faster searching.
4. Sets allow quick lookup of elements compared to normal lists.
5. The new program should produce the same output but run more efficiently.

Code:

```
19
20 names = ["Alice", "Bob", "Charlie", "David"]
21 search_names = ["Charlie", "Eve", "Bob"]
22
23 name_set = set(names)
24
25 for s in search_names:
26     if s in name_set:
27         print(s, "is in the list")
28     else:
29         print(s, "is not in the list")
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\Desktop\AI Assistanting coding\lab-13.py
Charlie is in the list
Eve is not in the list
Bob is in the list
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation

1. A set is created from the names list for faster searching.
2. The program checks each name using in operator.
3. This removes the need for nested loops.
4. The program becomes faster and easier to read.

Task 3 – Extracting Reusable Functions

Prompt:

1. The legacy program calculates price, tax, and total amount many times in the code.
2. Writing the same logic repeatedly makes the program harder to maintain.
3. Refactor the program by creating a reusable function called calculate_total.
4. The function should accept price as input and calculate tax and total price.
5. The program should call this function for different prices instead of repeating the same code.

Code:

```
32 def calculate_total(price):
33     """
34     This function calculates tax and total price
35     """
36     tax = price * 0.18
37     total = price + tax
38     print("Total Price:", total)
39
40
41 calculate_total(250)
42 calculate_total(500)
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/rahul/Desktop/Assistanting coding/lab-13.py"
Total Price: 295.0
Total Price: 590.0
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. A function calculate_total() is created to perform tax calculation.
2. The function takes price as input.

3. It calculates tax and adds it to the price.
4. This avoids repeating the same calculation code.

Task 4 – Replacing Hardcoded Values with Constants

Prompt:

1. The given program calculates the area and circumference of a circle using hardcoded numbers.
2. Hardcoded values make the program difficult to modify later.
3. Refactor the program by defining constants such as PI and RADIUS.
4. Use these constants in calculations instead of writing numbers directly.
5. This makes the program cleaner and easier to maintain.

Code:

```
46  PI = 3.14159
47  RADIUS = 7
48
49  area = PI * (RADIUS ** 2)
50  circumference = 2 * PI * RADIUS
51
52  print("Area of Circle:", area)
53  print("Circumference of Circle:", circumference)
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Python\Python314\python
Assistanting coding/lab-13.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\rahul\Desktop\AI Assistanting coding>
```

Explanation:

1. Constants PI and RADIUS are defined at the top of the program.
2. These constants replace hardcoded numbers.
3. The calculations now use these named constants.
4. This improves readability and maintainability.

Task 5 – Improving Variable Naming and Readability

Prompt

1. The given program uses unclear variable names such as a, b, and c.
2. Poor variable names make the program difficult to understand.
3. Refactor the code using meaningful variable names.
4. Add comments to explain the purpose of the calculation.
5. Ensure the output remains the same after refactoring.

Code:

```
55     ...  
56     base = 10  
57     height = 20  
58  
59     # Calculate area of triangle  
60     area = base * height / 2  
61  
62     print(area)
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & C:\Users\rahul\AppData\Local\Programs\Python\Python314\python.exe  
Assistanting coding/lab-13.py  
100.0  
PS C:\Users\rahul\Desktop\AI Assistanting coding> |  
Ln 62, Col 12  Spaces: 4  UTF-8  CRLF
```

Explanation:

1. Variable names are changed to base and height for better clarity.
2. A comment is added to explain the calculation.
3. The program calculates the area of a triangle.
4. The output remains the same but the code is easier to understand.